

FATE: Monitoring DIC

2025 Early Summer Samples

2026-05-22

Contents

0.1	Import Data Functions	2
0.2	Import Sample Data	2
0.3	Assessing Standard Curves	3
0.4	CRM Check - Don't run chunk if no CRMs run	4
0.5	Assess Check Standards	5
0.6	Assess Blanks	6
0.7	Sample Flagging - Are samples Within the range of the curve?	7
0.8	Visualize Data by Dock & Flume & Date	8
0.9	Convert data from mg/L to uMoles/L	9
0.10	Export Processed Data	9
0.11	Assess Duplicates - none on this run	9

```
##Setup - Change things here & write any notes
```

```
#identify section  
cat("Setup Information")
```

```
## Setup Information
```

```
##### Run information - PLEASE CHANGE  
Date_Run = "5/26/25" #Date that instrument was run  
Run_by = "Stephanie J. Wilson" #Instrument user  
Script_run_by = "Stephanie J. Wilson" #Code user  
run_notes = "No duplicates were included with this run." #any notes from the run  
samples <- c("FATE") #whatever identifies your samples within the same names  
samples_pattern <- paste(samples, collapse = "|")  
  #samples_pattern <- "GCW" #use this instead of the line above if you have only one site code  
chks_name = "Chk_Std_" #what did you name your check standards?  
crm_name = "CRM|crm" #what did you name your CRMS?  
  
##### File Names - PLEASE CHANGE  
#file path and name for raw summary data file  
raw_file_name = "Raw Data/FATE_DIC_202507.txt"  
  
#file path and name for raw all peaks file  
raw_allpeaks_name = "Raw Data/FATE_DIC_202507_allpeaks.txt"  
  
#file path and name of processed data file  
processed_file_name = "Processed Data/FATE_Processed_DIC_202507.csv"  
  
##### Log Files - PLEASE CHECK  
#downloaded metadata csv - downloaded from Google drive as csv for this year  
  #Raw_Metadata = "Raw Data/COMPASS_SynopticCB_PW_SampleLog_2025.csv"  
  
#qaqc log file path for this year  
Log_path = "Raw Data/FATE_DIC_QAQClog_2025.csv"
```

```
##Set Up Code
```

0.1 Import Data Functions

0.2 Import Sample Data

```
## Import Sample Data
```

```
## New names:  
## * ' -> '...14'
```

```
## Warning: One or more parsing issues, call 'problems()' on your data frame for details,  
## e.g.:  
##   dat <- vroom(...)  
##   problems(dat)
```

```
## # A tibble: 6 x 3
##   sample_name      ic_raw run_datetime
##   <chr>          <dbl> <chr>
## 1 FATE_Dock_20250529_A    12.9 7/25/2025 5:11:23 PM
## 2 FATE_Dock_20250529_B    12.9 7/25/2025 5:22:59 PM
## 3 FATE_Flume_20250529_A    13.3 7/25/2025 5:35:12 PM
## 4 FATE_Flume_20250529_B    13.3 7/25/2025 5:47:09 PM
## 5 FATE_Dock_20250605_A    13.5 7/25/2025 5:59:22 PM
## 6 FATE_Dock_20250605_B    13.4 7/25/2025 6:11:36 PM
```

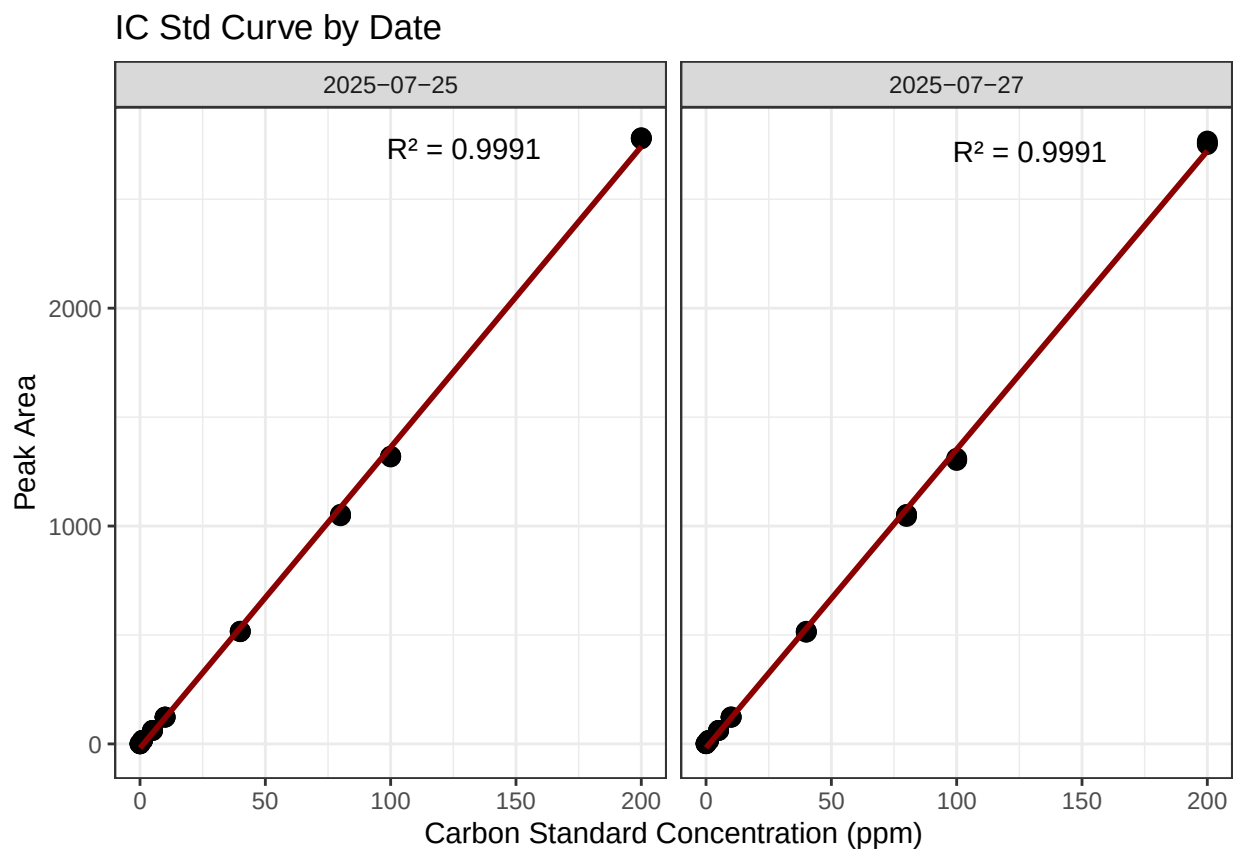
0.3 Assessing Standard Curves

```
## Assess the Standard Curves
```

```
## New names:
## * ' ' -> '...18'
```

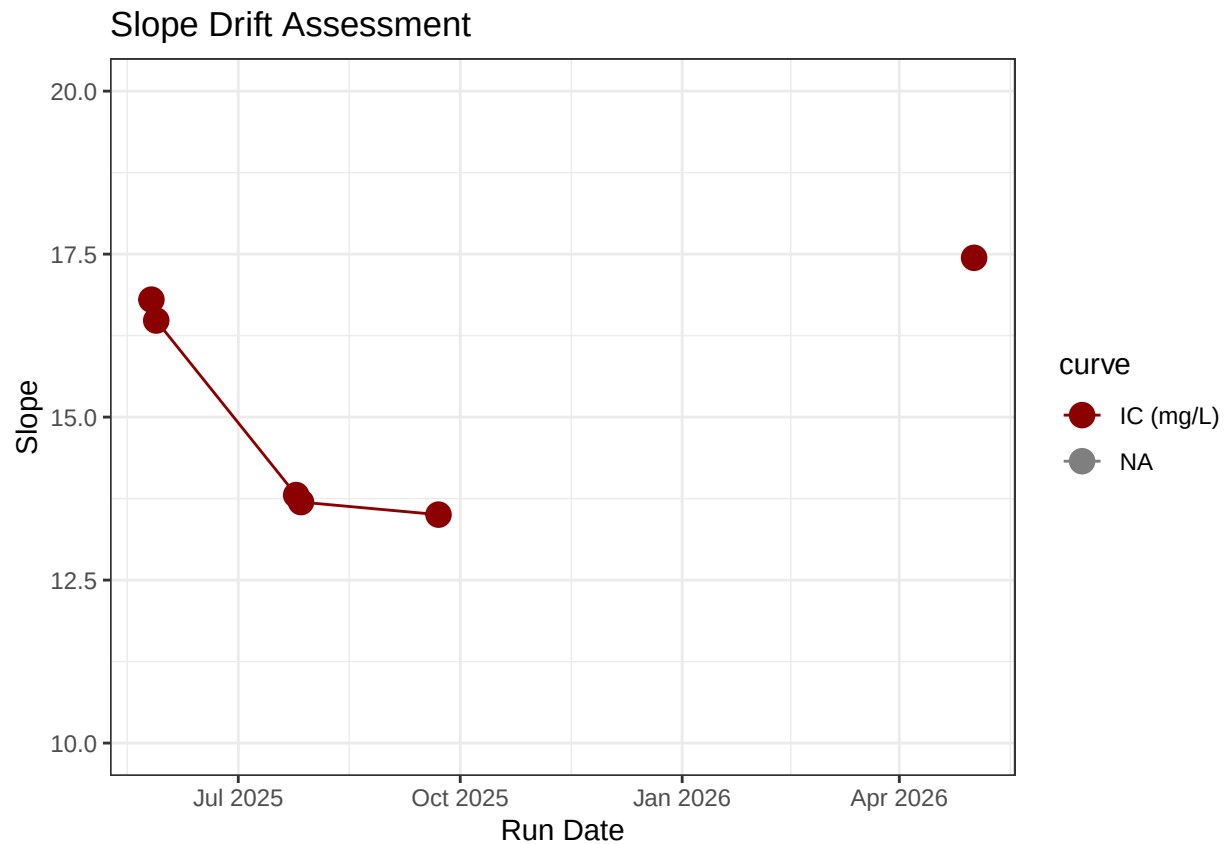
```
## Warning: One or more parsing issues, call 'problems()' on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## ('geom_point()').
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## ('geom_line()').
```



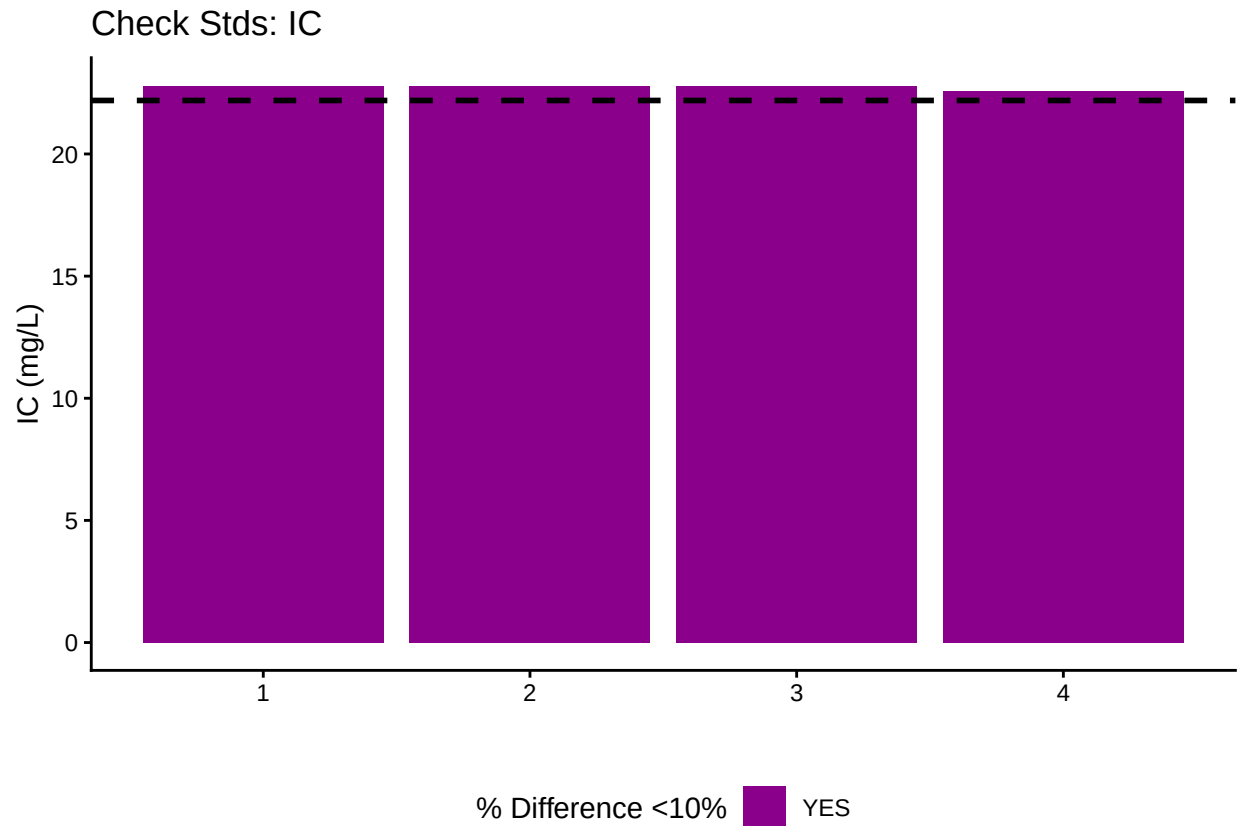
```
## [1] "IC Curve r2 GOOD"
```

0.4 CRM Check - Don't run chunk if no CRMs run

```
## Assess the CRMs
```

```
## New names:
## * '' -> '...14'
```

```
## Warning: One or more parsing issues, call 'problems()' on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```



```
## [1] "IC crm has a % Difference <25% of expected - PROCEED"
```

```
## Run mean = 22.7325
```

```
## Expected = 22.19
```

0.5 Assess Check Standards

```
## Assess the Check Standards
```

```
## New names:
```

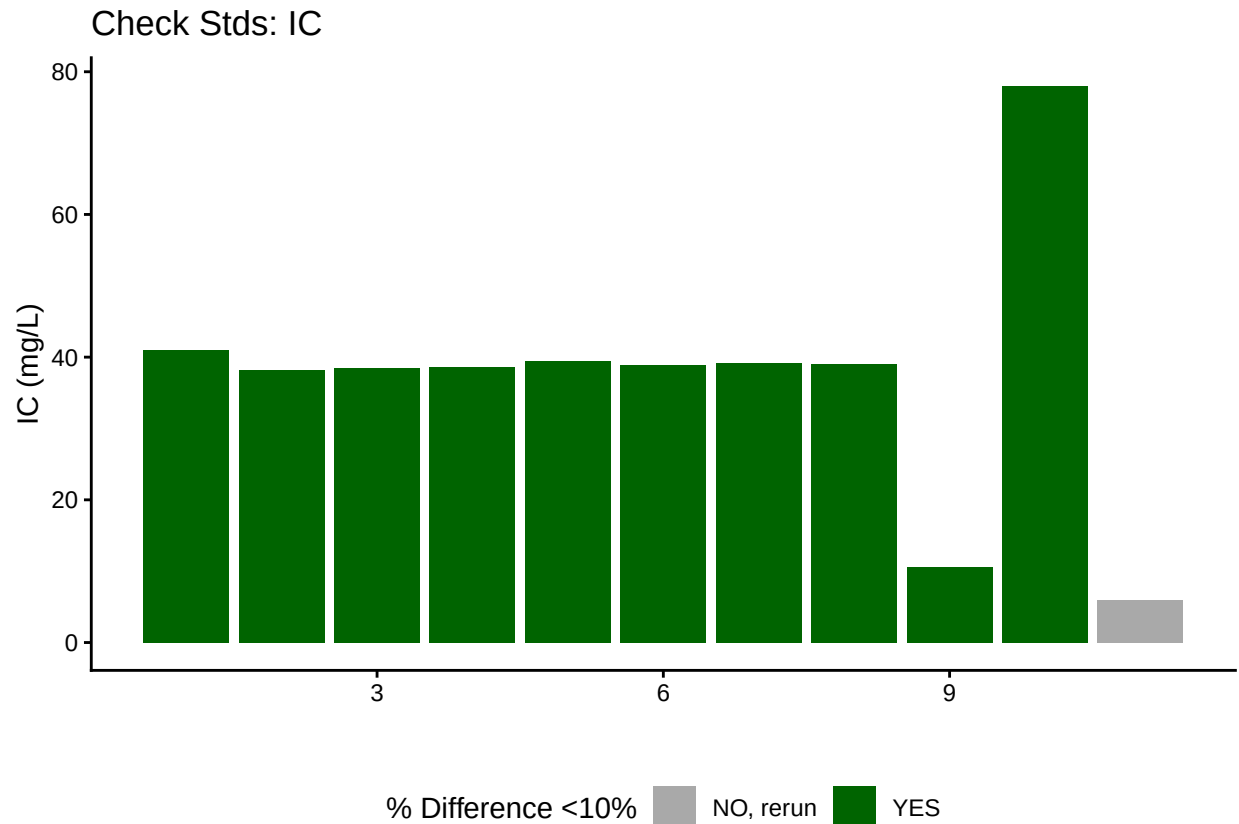
```
## * ' ' -> '...14'
```

```
## Warning: One or more parsing issues, call 'problems()' on your data frame for details,
```

```
## e.g.:
```

```
## dat <- vroom(...)
```

```
## problems(dat)
```



```
## [1] ">60% of IC Check Standards are within range of expected concentration"
```

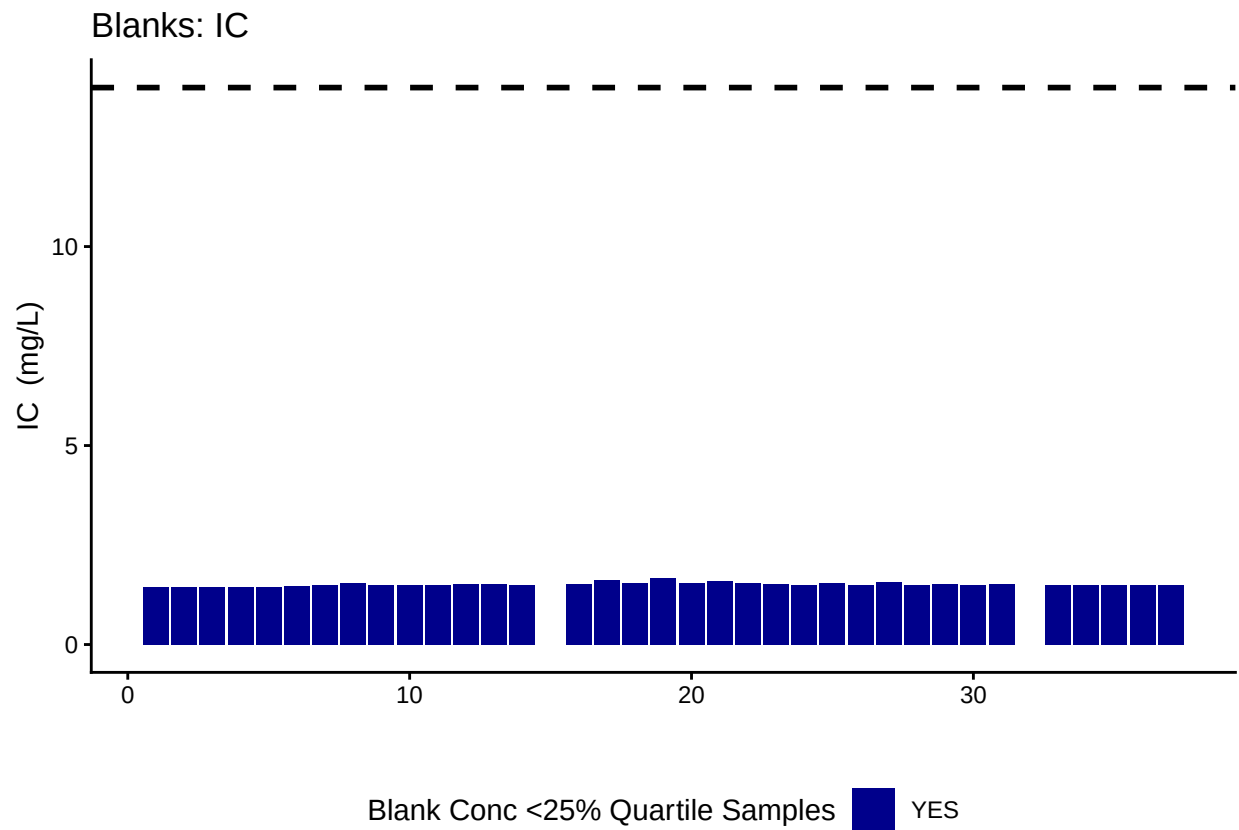
0.6 Assess Blanks

```
## Assess Blanks
```

```
## New names:
## * ' ' -> '...14'
```

```
## Warning: One or more parsing issues, call 'problems()' on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## [1] ">60% of Carbon Blank concentrations are lower 25% quartile of samples"
```

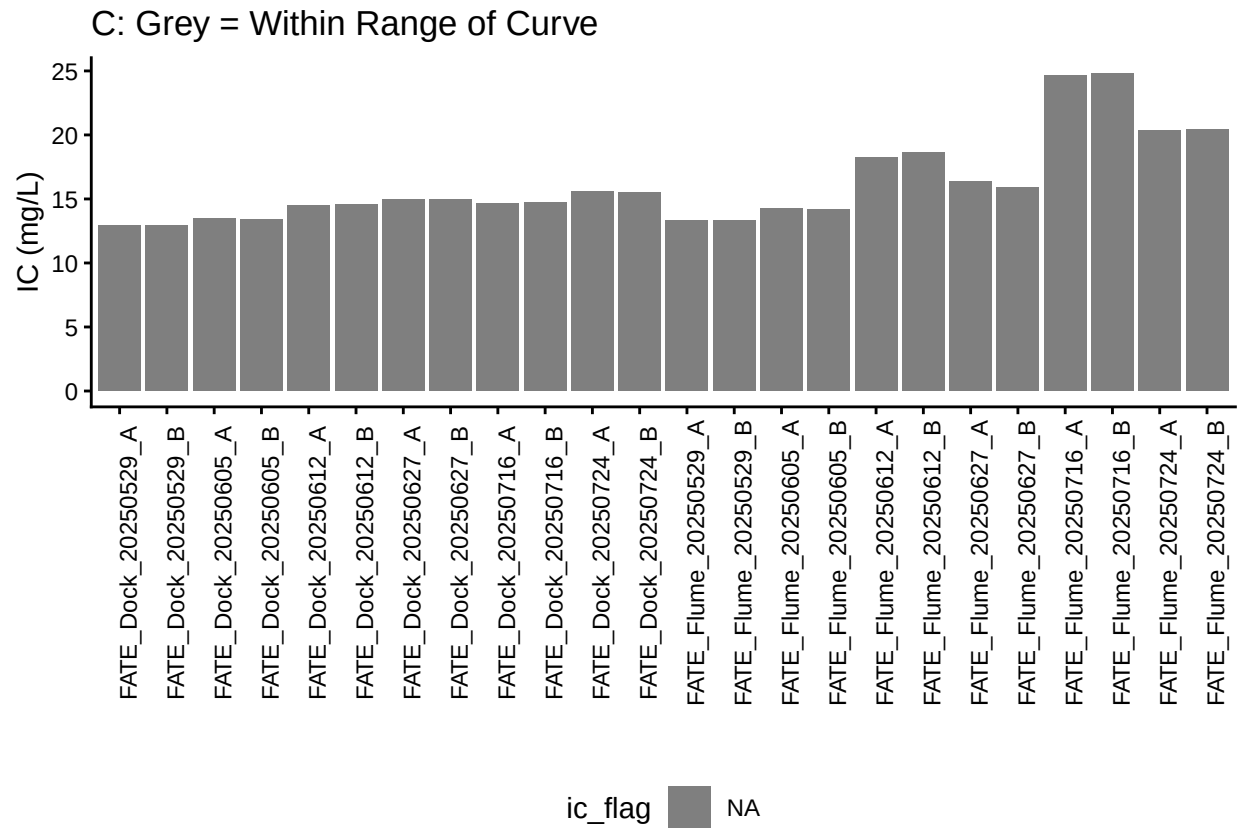


```
## carbon blanks:
```

```
## [1] 1.510657
```

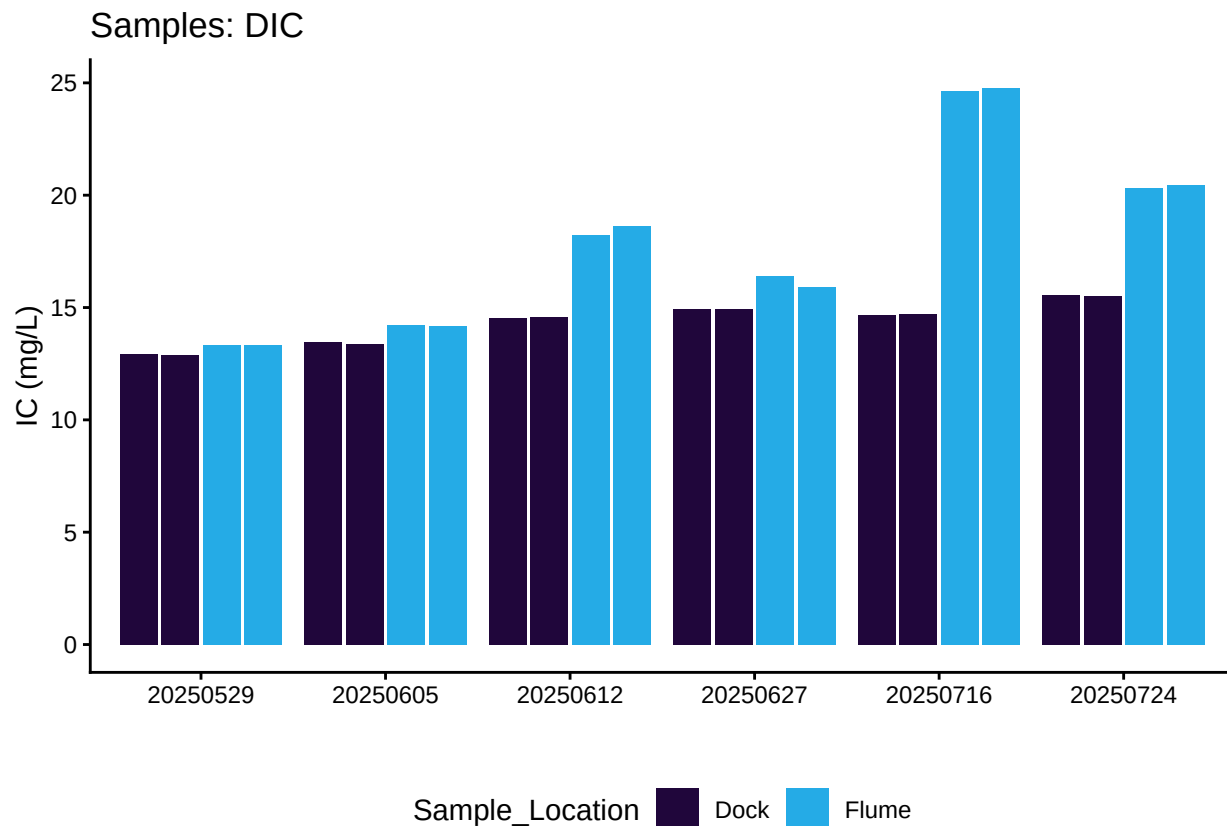
0.7 Sample Flagging - Are samples Within the range of the curve?

```
## Sample Flagging
```



0.8 Visualize Data by Dock & Flume & Date

Visualize Data



0.9 Convert data from mg/L to uMoles/L

0.10 Export Processed Data

Export Processed Data

```
## # A tibble: 6 x 9
##   Project Sample_Location Replicate Date   ic_mgL ic_uM ic_flag Analysis_runtime
##   <chr>    <chr>          <chr>   <chr>   <dbl> <dbl> <chr>   <chr>
## 1 FATE    Dock             A      20250~ 12.9 1076. ""      7/25/2025 5:11:~
## 2 FATE    Dock             B      20250~ 12.9 1073. ""      7/25/2025 5:22:~
## 3 FATE    Flume            A      20250~ 13.3 1110. ""      7/25/2025 5:35:~
## 4 FATE    Flume            B      20250~ 13.3 1109. ""      7/25/2025 5:47:~
## 5 FATE    Dock             A      20250~ 13.5 1121. ""      7/25/2025 5:59:~
## 6 FATE    Dock             B      20250~ 13.4 1112. ""      7/25/2025 6:11:~
## # i 1 more variable: Run_notes <chr>
```

0.11 Assess Duplicates - none on this run

```
“{#r Check Duplicates, echo=FALSE}
cat(“Assess Duplicates”)
#Take a look at the raw data #head(dat_raw)
```

```

#pull out any rows that have "dup" in the sample_name column
dups <- dat_raw %>%
select(!c(ic_flag)) %>% filter(str_detect(sample_name, "dup")) #have to change this to match data

#create a new dataframe and remove dups from sample dataframe
dat_raw2 <- dat_raw %>%
filter(!str_detect(sample_name, "dup"))

#remove the dup from these IDs so we will have duplicate sample names
dupssample_name <- gsub("dup", "", as.character(dupssample_name))
dups <- dups[, -c(3)] #remove the run date time
for(colnames(dups) <- c('sample_name', 'ic_raw_dup'))

#merge with the dataframe so we have a column for the conc and the dup
QAdups <- merge(dat_raw2, dups)

#create a dataframe to compare npoc dups
df2 <- as.data.frame(QAdups[, c("raw", "dup")])
df2dups <- QAdups[, c("raw", "dup")]

#calculate the cv of the duplicates
df2sds <- apply(df2, 1, sd)
df2mean <- apply(df2, 1, mean)

QAdups[, c("sds", "mean")] <- (df2sds/df2mean) * 100
QAdups[, c("cv", "flag")] <- ifelse(QAdups[, c("sds", "mean")] < 10,
'YES', 'NO, rerun')

#Put all the dups together and create row numbers for plotting
QAdups <- QAdups %>% mutate(row_num = row_number()) #head(QAdups)

#plot dups output as a bar graph to easily check - want any over 10% to be red need to work on this
C_dups <- ggplot(data = QAdups, aes(x = row_num, y = ic_dups_cv, fill = ic_dups_cv_flag)) +
geom_bar(stat = 'identity') + theme_classic() + labs(x = "", y = "CV of IC Duplicates") +
scale_fill_manual(values = c("YES" = "darkgreen", "NO, rerun" = "red")) +
theme(legend.position = "none") + geom_hline(yintercept = 10, linetype = "dashed", color = "black", linewidth = 1) +
guides(fill = guide_legend(title = "CV Between Dups < 10%"))

C_dups

#calculate the percent of check standards that are within the range based on the flag
c_dups_percent <- (sum(QAdups[, c("cv", "flag")] == "YES")/nrow(QAdups))*100

#report out if the dups are within range
ifelse(c_dups_percent >= chks_flag, ">60% of Carbon Duplicates have a CV < 10%",
"<60% of Carbon Duplicates have a CV < 10% - REASSESS")

#write out a flag to the sample dataframe if more than 60% of the dups have CVs out of range
if (c_dups_percent <= chks_flag) {
dat_raw[, c("flag", "ic_flag")] <- ifelse(dat_raw[, c("flag", "ic_flag")] != "",
paste0(dat_raw[, c("flag", "ic_flag")], "IC dups out of range"), "IC dups out of range")
}

"

```